

A CLOUD-POWERED RETAIL E-COMMERCE SALES ANALYTICS

CLOUD COMPUTING PROJECT REPORT

Submitted by

SACHIN N (230701274)

SAI ARAVIND S (230701275)

SARVESH R (230701295)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE NOVEMBER 2025**

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled “A CLOUD-POWERED RETAIL E-COMMERCE SALES ANALYTICS” is the bonafide work of **SACHIN N (230701274), SAI ARAVIND S (230701275) and SARVESH R (230701295)** who carried out the work under my supervision. Certified further that, to the best of my knowledge, the work reported herein does not form part of any other thesis or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy Supervisor
Head of The Department
Department of Computer Science and
Engineering,
Rajalakshmi Engineering College.

SIGNATURE

Dr.Muneeshwari Radhakrishnan
Professor & Associate Professor,
Department of Computer Science and
Engineering ,
Rajalakshmi Engineering College

Submitted to the Project Viva Voce Examination held on _____.

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to the Almighty for His blessings and guidance through-out the successful completion of this project. We extend our heartfelt thanks to our Chairman **Mr. S. Meganathan, B.E., F.I.E.**, Vice Chairman **Mr. Abhay Shankar Meganathan, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) Thangam Meganathan, Ph.D.**, for providing excellent infrastructure and academic support.

We sincerely thank **Dr. S.N. Murugesan, M.E., Ph.D.**, Principal of Rajalakshmi Engineering College, for his constant encouragement and support. We are deeply grateful to **Dr. E.M. Malathy, Ph.D.**, Head of the Department of Computer Science and Engineering, for her valuable guidance and encouragement throughout the project.

We express our sincere gratitude to our project guide **Dr. Muneeshwari Radhakrishnan, M.E., Ph.D.**, Associate Professor, Department of Computer Science and Engineering, for her continuous support, insightful suggestions, and encouragement during every phase of the project.

Finally, we thank our friends, classmates, and family members for their motivation and support throughout the development of this project.

ABSTRACT

The rapid advancement of cloud computing and web technologies has significantly transformed the e-commerce industry by enabling scalable, reliable, and accessible online platforms. Modern businesses increasingly require cloud-based solutions that provide flexibility, high availability, centralized management, and seamless deployment capabilities. Traditional locally hosted web applications often face challenges such as limited scalability, deployment complexity, infrastructure maintenance, and reduced accessibility.

The primary objective of this project is to develop and deploy a cloud-based e-commerce web application using Microsoft Azure. The system is designed to provide a scalable and efficient online platform capable of handling web requests, database operations, and cloud deployment processes through modern cloud-native technologies. The application integrates frontend interfaces, backend services, and cloud infrastructure into a unified environment that improves reliability, maintainability, and deployment efficiency.

The application frontend is developed using HTML templates and responsive web design techniques, while the backend services are implemented using Python Flask. Azure SQL Database is used for storing application data, and Microsoft Azure App Service is used for cloud hosting and deployment. The system utilizes Gunicorn as the production server for handling Flask application requests within the Linux runtime environment provided by Azure.

The deployment workflow incorporates GitHub integration, ZIP deployment methods, startup command configuration, and cloud runtime management. Various deployment challenges such as module import errors, runtime configuration issues, Gunicorn setup problems, and dependency management were identified and resolved during the implementation process. The project demonstrates practical cloud deployment concepts including scalability, platform-as-a-service (PaaS) hosting, centralized infrastructure management, and production-grade application deployment.

Inference: The adoption of a cloud-native deployment architecture enables improved scalability, resource optimization, and high availability compared to traditional local hosting environments. The separation of frontend, backend, and cloud infrastructure components improves modularity and maintainability. Azure App Service reduces infrastructure management overhead while providing secure and reliable hosting services. The use of Flask and Gunicorn enables lightweight and efficient backend processing, while Azure SQL Database ensures secure and scalable data management. The overall system demonstrates the practical implementation of cloud computing technologies for modern web application deployment and management.

Contents

ACKNOWLEDGEMENT	2
ABSTRACT	3
	Page No
1 INTRODUCTION	6
1.1 Overview	6
1.2 Motivation	6
1.3 Objectives	6
1.4 Scope of the Project	7
1.5 Project Outcomes & Inferences	7
2 LITERATURE SURVEY	8
2.1 Existing Systems	8
2.2 Limitations of Existing Systems	8
2.3 Proposed System	9
3 SYSTEM ANALYSIS	10
3.1 Problem Statement	10
3.2 Feasibility Study	10
3.2.1 Technical Feasibility	10
3.2.2 Economic Feasibility	10
3.2.3 Operational Feasibility	11
3.3 Software Requirements	11
3.4 Hardware Requirements	11
3.5 Requirement Analysis & Key Inferences	12
4 SYSTEM DESIGN	13
4.1 System Architecture	13
4.2 Modules of the System	13
4.3 UML Diagrams	14
4.3.1 Use Case Diagram	14
4.3.2 Class Diagram (Backend Models)	14
4.3.3 Sequence Diagram (File Upload Flow Example)	15

4.3.4	Data Flow Diagram (Level 0)	15
5	SYSTEM IMPLEMENTATION	16
5.1	Frontend Development	16
5.2	Backend Development.....	16
5.3	Database Implementation	16
5.4	Authentication and Security	17
5.5	API Integration and External Services.....	17
6	CLOUD DEPLOYMENT AND DEVOPS	18
6.1	Cloud Computing Overview.....	18
6.2	Microsoft Azure Services Used	18
6.3	Containerization	18
6.4	CI/CD Pipeline (GitHub Actions)	18
6.5	Monitoring and Logging	19
6.6	Scalability and Reliability	19
7	TESTING AND RESULTS	20
7.1	Testing Methodology	20
7.2	Detailed Test Cases and Results.....	20
7.3	Performance Analysis and Inferences	21
7.4	User Acceptance Testing Feedback	21
7.5	Security Testing Summary	22
8	CONCLUSION AND FUTURE ENHANCEMENTS	23
8.1	Conclusion	23
8.2	Limitations	23
8.3	Future Enhancements.....	24
9	REFERENCES	25
A	Appendices	26
A.1	Screenshots	26
A.2	Source Code Repository	26
A.3	Deployment URL.....	26
A.4	API Documentation (Sample Endpoints)	26
A.5	Deployment Architecture Diagram.....	27
A.6	Project Timeline (Gantt Summary)	27

CHAPTER 1

INTRODUCTION

1.1 Overview

The rapid growth of cloud computing and web technologies has significantly transformed the modern e-commerce industry. Businesses today increasingly rely on cloud-based applications to provide scalable, reliable, and globally accessible online services. Traditional locally hosted systems often face challenges such as infrastructure limitations, deployment complexity, scalability issues, and high maintenance costs. Cloud computing provides an effective solution to these problems by enabling flexible resource management, centralized deployment, and high availability.

The proposed project, **Cloud-Based E-Commerce Web Application using Microsoft Azure**, is designed to demonstrate the practical implementation of cloud-native web application deployment using modern technologies and cloud services. The application provides a centralized online platform capable of handling web requests, database operations, and cloud hosting efficiently through Microsoft Azure infrastructure.

The system integrates frontend interfaces, backend services, and cloud deployment components into a unified architecture. The frontend is developed using HTML templates and responsive web technologies, while the backend services are implemented using Python Flask. Microsoft Azure App Service is used for cloud deployment and hosting, and Azure SQL Database is used for secure cloud-based data storage and management.

The application deployment process includes runtime configuration, startup command management, cloud infrastructure setup, and production server integration using Gunicorn. The project demonstrates real-world cloud deployment concepts such as scalability, reliability, centralized infrastructure management, and production-grade application hosting.

1.2 Motivation

Modern businesses require highly available and scalable web applications that can handle increasing user demands without infrastructure limitations. Traditional deployment methods using local servers often result in performance bottlenecks, reduced accessibility, and complicated maintenance procedures. Additionally, managing application hosting, server configuration, runtime environments, and deployment pipelines manually increases operational complexity.

Cloud platforms such as Microsoft Azure provide an efficient alternative by offering Platform-as-a-Service (PaaS) solutions that simplify deployment and infrastructure management. Azure App Service allows developers to deploy applications quickly without manually configuring physical servers or managing low-level infrastructure components.

The motivation behind this project is to gain practical experience in cloud-native application deployment and demonstrate how modern cloud technologies can simplify web application hosting

and management. The project focuses on understanding real-world deployment challenges such as runtime configuration issues, startup command management, dependency installation, server configuration, and cloud troubleshooting.

Another motivation is to implement a scalable e-commerce platform capable of remote accessibility, centralized management, and efficient deployment using cloud computing technologies. The project also aims to provide exposure to DevOps concepts, production deployment workflows, and cloud service integration.

1.3 Objectives

The major objectives of the project are:

- To develop a cloud-based e-commerce web application accessible through the internet.
- To implement backend services using Python Flask.
- To deploy the application using Microsoft Azure App Service.
- To configure and manage cloud runtime environments effectively.
- To integrate Azure SQL Database for cloud-based data storage.
- To implement scalable cloud hosting infrastructure using Azure services.
- To configure Gunicorn as the production server for Flask deployment.
- To understand deployment troubleshooting techniques in cloud environments.
- To implement centralized hosting and remote accessibility.

1.4 Scope of the Project

The project focuses on developing and deploying a cloud-hosted e-commerce web application using Microsoft Azure cloud services. The application is accessible through web browsers and can be hosted remotely using Azure infrastructure.

1.5 Project Outcomes & Inferences

The project successfully demonstrates the deployment of a cloud-based e-commerce application using Microsoft Azure cloud services. The implementation provides practical understanding of cloud infrastructure management, application hosting, and deployment workflows.

The project provides valuable exposure to practical cloud computing concepts, DevOps workflows, Linux runtime management, production server configuration, and cloud deployment troubleshooting techniques widely used in modern software industries.

CHAPTER 2 LITERATURE SURVEY

2.1 Existing Systems

A study of currently available e-commerce and cloud hosting solutions reveals that many businesses rely on separate tools and platforms for frontend hosting, backend deployment, database management, and cloud infrastructure configuration. While these platforms provide powerful individual functionalities, they often require complex integration and management processes.

Some commonly used existing systems include:

- **Traditional Local Hosting Systems:** Many small-scale applications are hosted on local servers or shared hosting platforms. These systems suffer from limited scalability, poor reliability, and high maintenance overhead.
- **AWS Cloud Hosting:** Amazon Web Services provides cloud deployment services such as EC2, Elastic Beanstalk, and RDS for hosting web applications and databases. However, configuration complexity can be challenging for beginners.
- **Google Cloud Platform (GCP):** GCP provides cloud-based hosting and database services with scalability features, but requires advanced cloud configuration knowledge.
- **Microsoft Azure App Service:** Azure provides Platform-as-a-Service (PaaS) hosting solutions for deploying web applications without managing physical infrastructure. It supports multiple programming languages and cloud database integration.
- **Heroku:** A cloud application platform that simplifies deployment processes for web applications. However, limited free resources and scalability constraints affect large-scale deployment.
- **DigitalOcean Droplets:** Provides virtual machine-based cloud hosting services but requires manual server configuration and maintenance.
- **Traditional E-Commerce Platforms:** Platforms such as Shopify and WooCommerce provide ready-made e-commerce functionality but offer limited flexibility for custom backend development and cloud infrastructure learning.

Most existing systems provide hosting or e-commerce functionality independently, but they often lack simplified deployment workflows, centralized cloud management, and beginner-friendly infrastructure configuration for educational cloud projects.

2.2 Limitations of Existing Systems

The following limitations were identified in existing systems and deployment approaches:

- **Complex Deployment Configuration:** Many cloud platforms require extensive manual configuration for servers, runtime environments, and dependencies.
- **Infrastructure Management Overhead:** Traditional hosting systems require manual server maintenance, updates, and monitoring.
- **Scalability Challenges:** Local hosting environments struggle to handle increasing user traffic and application scaling.

- **Dependency Management Issues:** Improper package installation and runtime mismatches often cause deployment failures.
- **Production Server Configuration Problems:** Configuring servers such as Gunicorn, Nginx, or Apache can be difficult for beginners.
- **Limited Accessibility:** Locally hosted applications cannot provide reliable global accessibility.
- **Deployment Troubleshooting Complexity:** Errors related to runtime environments, startup commands, module imports, and cloud configurations require technical expertise.
- **High Maintenance Cost:** Managing dedicated infrastructure increases operational expenses.
- **Lack of Centralized Cloud Management:** Traditional systems require separate management of servers, databases, networking, and deployment pipelines.
- **Security and Reliability Concerns:** Local systems may lack enterprise-grade security and backup capabilities available in cloud environments.

2.3 Proposed System

The proposed system is a Cloud-Based E-Commerce Web Application deployed using Microsoft Azure Cloud Services. The application is designed to provide scalable, reliable, and centralized web hosting through cloud-native infrastructure.

The system integrates frontend interfaces, backend services, database connectivity, and cloud deployment components into a unified architecture. The backend is developed using Python Flask, while Microsoft Azure App Service is used for deployment and hosting. Azure SQL Database provides cloud-based data storage and management.

The application uses Gunicorn as the production WSGI server for running Flask applications in the Azure Linux environment. Cloud deployment is achieved through Azure App Service deployment pipelines and runtime configuration management.

Key Features of the Proposed System

- Centralized cloud-based application hosting using Microsoft Azure.
- Scalable web application deployment through Azure App Service.
- Python Flask backend integration with cloud runtime environments.
- Secure cloud database management using Azure SQL Database.
- Production-ready deployment using Gunicorn server.
- Simplified infrastructure management through Platform-as-a-Service (PaaS).
- Remote accessibility from any internet-connected device.
- Reduced hardware dependency through cloud hosting.
- Runtime environment management and deployment automation.
- Improved reliability, scalability, and maintainability.

CHAPTER 3 SYSTEM ANALYSIS

3.1 Problem Statement

Modern web applications and e-commerce systems require scalable, reliable, and easily accessible hosting environments. Traditional local hosting methods often face limitations such as poor scalability, complex maintenance procedures, hardware dependency, and deployment difficulties. Managing servers, configuring runtime environments, and ensuring continuous availability become challenging as application complexity increases.

During cloud deployment, developers frequently encounter issues related to runtime configuration, dependency management, startup command setup, and production server integration. Errors such as module import failures, missing dependencies, incorrect startup commands, and server boot failures can interrupt deployment workflows and reduce application reliability.

Additionally, local deployment environments lack centralised cloud management, automatic scalability, or enterprise-grade infrastructure support. Businesses and developers require cloud-native solutions that simplify deployment, improve accessibility, and reduce operational overhead.

The major problems addressed in this project include:

- Difficulty in deploying scalable web applications using traditional hosting methods.
- Complex cloud runtime configuration and dependency management.
- Challenges in configuring production servers such as Gunicorn.
- Limited accessibility and scalability in local hosting environments.
- Lack of centralized infrastructure management.
- Deployment failures caused by incorrect startup commands and module import issues.
- Infrastructure maintenance overhead in traditional systems.
- Requirement for secure and reliable cloud database integration.
- Need for practical understanding of cloud deployment workflows and troubleshooting.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The technologies used in the project such as Python Flask, Microsoft Azure App Service, Azure SQL Database, and Gunicorn are modern, industry-standard, and well documented. Flask provides a lightweight and flexible backend framework suitable for cloud deployment, while Azure App Service simplifies hosting and runtime management through Platform-as-a-Service (PaaS) architecture.

Azure SQL Database offers secure and scalable cloud-based database management without requiring manual server maintenance. Gunicorn acts as a production-ready WSGI server capable of efficiently handling Flask application requests within Linux environments.

The development and deployment process is technically feasible because all required technologies provide extensive documentation, community support, and cloud integration capabilities. The project

also demonstrates compatibility between frontend interfaces, backend services, and cloud infrastructure components.

Inference

The combination of Flask, Azure App Service, and Azure SQL Database forms an efficient cloud-native architecture suitable for modern web application deployment. Azure's managed services reduce infrastructure complexity while improving deployment reliability and scalability. The learning curve remains manageable due to extensive documentation and community support available for these technologies.

3.2.2 Economic Feasibility

The project is economically feasible because Microsoft Azure provides cost-effective cloud hosting plans and student subscription benefits through Azure for Students. The pay-as-you-go pricing model reduces infrastructure investment and allows efficient resource utilization based on application demand.

Azure App Service enables hosting without purchasing dedicated hardware or maintaining physical servers. Azure SQL Database also eliminates the cost of managing database infrastructure manually. Since the application is developed using open-source technologies such as Python Flask, additional software licensing costs are minimized.

Inference

The cloud deployment cost can be maintained at a minimal level using Azure student subscriptions and entry-level hosting plans. The use of Platform-as-a-Service infrastructure significantly reduces hardware investment, maintenance costs, and operational expenses, making the project economically sustainable.

3.3 Software Requirements

- Operating System: Windows 10/11, Linux, or macOS for development purposes.
- Frontend Technologies: HTML5, CSS3, JavaScript, Bootstrap.
- Backend Framework: Python Flask.
- Database: Microsoft Azure SQL Database.
- Cloud Platform: Microsoft Azure App Service.
- Production Server: Gunicorn.
- Development Environment: Visual Studio Code.
- Version Control: Git and GitHub.
- Deployment Platform: Azure App Service Deployment Center / VS Code Azure Extension.
- Runtime Environment: Python 3.11+.
- Database Connectivity: pyodbc.
- API Communication: REST-based communication between frontend and backend.

3.4 Hardware Requirements

- Development System:
 - Minimum 8 GB RAM
 - Intel i5 processor or equivalent
 - 20 GB free storage
 - Stable internet connection
- Cloud Infrastructure:
 - Azure App Service Basic (B1) Plan
 - Azure SQL Database server
- End User Requirements:
 - Any device with internet connectivity
 - Modern web browser such as Chrome, Edge, or Firefox
 - Minimum 2 GB RAM

3.5 Requirement Analysis & Key Inferences

Functional Requirements

- User access through cloud-hosted web application.
- Backend API integration using Flask.
- Azure SQL Database connectivity for data retrieval and storage.
- Displaying database records through frontend dashboards.
- Cloud deployment using Azure App Service.
- Runtime environment configuration and startup command management.
- Production server integration using Gunicorn.
- Deployment monitoring and troubleshooting.
- Data visualization using frontend charts and tables.

Non-Functional Requirements

Performance

- Fast API response and efficient database connectivity.

Scalability

- Ability to scale application resources using Azure App Service plans.

Security

- Secure database connectivity and HTTPS-based communication.

Reliability

- Continuous cloud accessibility and centralized hosting.

Availability

- Remote access through internet-enabled devices.

Maintainability

- Modular architecture for easier updates and deployment management.

CHAPTER 4 SYSTEM DESIGN

4.1 System Architecture

The proposed E-Commerce Cloud Application follows a three-tier cloud architecture using Microsoft Azure services. The system integrates frontend interfaces, backend services, cloud hosting infrastructure, and database connectivity into a centralized cloud-native environment.

Frontend Layer

The frontend layer provides the user interface for accessing application features and visualizing e-commerce data. The frontend is developed using HTML, CSS, JavaScript, and Bootstrap-based responsive components. The interface communicates with backend APIs through HTTP requests.

The dashboard displays:

- Product data
- Sales information
- Dynamic tables
- Interactive charts
- Database-driven visualizations

The frontend is designed to be responsive and accessible from desktops, laptops, and mobile devices through modern web browsers.

Backend Layer

The backend layer is developed using Python Flask and handles:

- API routing
- Database connectivity
- Data processing
- Request handling
- JSON responses

Database Layer

The database layer uses Microsoft Azure SQL Database for storing e-commerce data such as:

- Product details
- Invoice information
- Quantity records
- Customer data
- Transaction information

The database is cloud-hosted and securely connected to the Flask backend using SQL authentication and encrypted cloud connectivity.

Cloud Infrastructure

The application is deployed using Microsoft Azure cloud services:

- **Azure App Service** – Hosts the Flask backend and web application.
- **Azure SQL Database** – Stores application and e-commerce data.
- **Azure Deployment Center** – Manages deployment workflows.
- **Azure Monitor & Log Stream** – Used for deployment monitoring and troubleshooting.
- **Gunicorn Server** – Handles production request processing.

4.2 Modules of the System

1. Frontend Dashboard Module

Displays:

- Product tables
- Database records
- Interactive charts
- Sales visualizations .

2. Backend API Module

Handles:

- API routing
- Database queries
- JSON response generation
- Request handling

3. Database Connectivity Module

Establishes secure connection between Flask backend and Azure SQL Database using:

- pyodbc
- SQL Server drivers
- Cloud authentication

4. Cloud Deployment Module

Handles:

- Azure App Service deployment
- Runtime configuration
- Startup command setup
- Production hosting

5. Data Visualization Module

Implements:

- Bar charts
- Product quantity analysis
- Dynamic chart rendering
- Interactive dashboard visualization

4.3 UML Diagrams

4.3.1 Use Case Diagram

Actors

- User
- Admin
- Azure Cloud Services

Use Cases

- View Dashboard
- Fetch Product Data
- Display Charts
- Connect to Database
- Deploy Application
- Monitor Logs
- Manage Cloud Hosting

4.3.2 Class Diagram

Classes

User

- userID
- username
- login()

Product

- productID
- description
- quantity
- unitPrice

DatabaseConnection

- connect()
- fetchData()

FlaskAPI

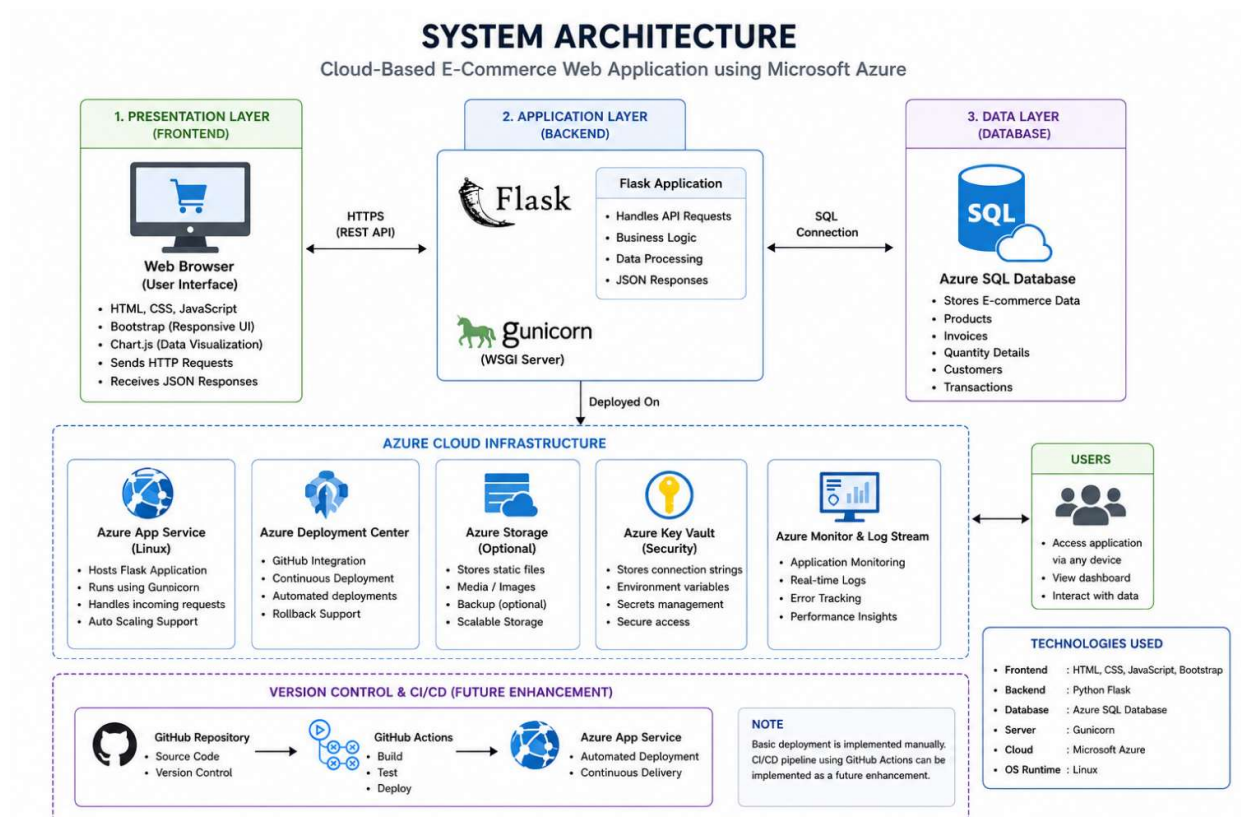
- getData()
- renderDashboard()

AzureDeployment

- deployApp()
- configureRuntime()

ChartVisualization

- generateChart()
- updateChart()



CHAPTER 5 SYSTEM IMPLEMENTATION

5.1 Frontend Development

The frontend of the cloud-based e-commerce web application is developed using HTML, CSS, JavaScript, Bootstrap, and Chart.js. The user interface is designed to provide a simple and responsive dashboard for viewing e-commerce data retrieved from the backend. The frontend includes pages for displaying product details, tabular records, and chart-based visualizations.

The dashboard page is implemented using an index.html file stored inside the templates folder of the Flask project. Flask renders this page through the /dashboard route. The frontend communicates with the backend using JavaScript fetch() requests to retrieve data from the /data API endpoint. The returned JSON data is dynamically displayed in an HTML table.

Chart.js is used to generate visual insights from the e-commerce dataset. Product descriptions and quantity values are processed in JavaScript to create charts such as top-selling product charts. The frontend also supports responsive layout behavior, allowing users to access the application through desktop or mobile browsers.

5.2 Backend Development

The backend of the application is developed using Python Flask. Flask is used to define routes, handle HTTP requests, connect with the database, and return JSON responses to the frontend. The backend acts as the core processing layer of the application.

The main backend file is app.py. It contains routes such as /, /data, and /dashboard. The / route is used to test whether the application is running successfully. The /data route connects to Azure SQL Database, executes SQL queries, retrieves e-commerce records, and returns them in JSON format. The /dashboard route renders the frontend template.

Database connectivity is implemented using the pyodbc library. The Flask backend connects to Azure SQL Database using the server name, database name, username, password, and SQL Server ODBC driver. Unicorn is used as the production WSGI server to run the Flask application on Azure App Service.

5.3 Database Implementation

Azure SQL Database is used as the primary database for the project. It stores structured e-commerce data such as invoice number, stock code, product description, quantity, invoice date, unit price, customer ID, and country. The table used in the project is ecommerce_table.

The backend retrieves records from Azure SQL Database using SQL queries. The /data API endpoint executes a query such as:

```
SELECT TOP 100 * FROM ecommerce_table;
```

The retrieved rows are converted into JSON format and sent to the frontend. This enables the dashboard to display live cloud database records.

Azure SQL Database provides managed database hosting, automatic backups, security, and scalability. Firewall rules were configured to allow access from the developer machine and Azure services.

5.4 Deployment and Cloud Configuration

The application is deployed using Microsoft Azure App Service with Linux runtime. Azure App Service provides a managed cloud hosting environment where the Flask application can run without manually managing physical servers.

The project files were deployed through VS Code Azure Extension and Azure deployment tools. The deployed folder structure included the Flask backend file, templates folder, and requirements.txt file. The requirements.txt file contains the required dependencies:

flask

pyodbc

gunicorn

A startup command was configured in Azure Stack Settings to run the Flask application using Gunicorn:

```
gunicorn ecommerce.app:app --bind 0.0.0.0:8000
```

The 0.0.0.0:8000 binding allows Azure App Service to route external requests to the Flask application. Azure Log Stream and Kudu SSH were used to monitor runtime errors and troubleshoot deployment issues.

5.5 Troubleshooting and Error Resolution

During deployment, several issues were encountered and resolved. Initially, the application failed due to incorrect module paths such as No module named app and No module named ecommerce. This was caused by mismatches between the deployed folder structure and the Gunicorn startup command.

Another major issue was missing dependencies such as No module named gunicorn and No module named pyodbc. These errors occurred because the requirements.txt file was not placed in the correct root directory or because the Azure build process did not install packages properly. The issue was resolved by correcting the file name and moving requirements.txt to the correct deployment root.

5.6 API Integration and Data Visualization

The backend API /data is integrated with the frontend dashboard. When the dashboard loads, JavaScript sends a request to the Flask API and receives e-commerce records in JSON format. The frontend then dynamically creates table rows and chart data from the API response.

The visualization logic groups products based on the Description field and calculates total quantity sold for each product. The top products are displayed using a bar chart. This allows users to understand product demand and sales distribution visually.

CHAPTER 6

6.1 Cloud Computing Overview

Cloud computing provides on-demand access to computing resources such as servers, databases, storage, and networking through the internet. It eliminates the need for managing physical hardware and allows applications to scale dynamically based on user demand. The e-commerce analytics project uses Microsoft Azure cloud services to host the application, database, and monitoring tools.

The project leverages Platform-as-a-Service (PaaS) solutions provided by Azure. Instead of manually configuring servers, Azure App Service automatically manages runtime environments, scaling, updates, and security patches. This reduces infrastructure complexity and allows developers to focus mainly on application development and deployment.

The cloud-based deployment also enables remote accessibility, meaning the application can be accessed from anywhere using a web browser. Azure additionally provides reliability, backup support, monitoring, and secure communication through HTTPS.

6.2 Microsoft Azure Services Used

The project uses multiple Microsoft Azure services for hosting, database connectivity, and monitoring.

Azure App Service

Azure App Service (Linux) is used to host the Flask-based backend application. It provides automatic runtime management, HTTPS support, deployment slots, and integrated monitoring features. The Flask application runs using Gunicorn inside the Azure Linux environment.

Azure SQL Database

Azure SQL Database is used to store e-commerce data in a structured relational format. It provides managed cloud database services with backup, firewall security, and scalability.

Azure Storage and Deployment Services

Azure deployment infrastructure automatically extracts and deploys uploaded application files into the runtime environment. Oryx build system is used internally by Azure for dependency installation and deployment configuration.

Azure Log Stream and Kudu

Azure Log Stream is used to monitor real-time runtime logs and identify deployment errors. Kudu SSH console allows developers to inspect deployment folders, edit files, and debug runtime issues directly inside the cloud environment.

Azure Monitor

Azure Monitor and diagnostic logs help track application failures, startup issues, and deployment status.

6.3 Deployment Process

The deployment process involved uploading the Flask application to Azure App Service through Visual Studio Code Azure Extension. The deployment package contained:

- app.py
- templates/
- requirements.txt

The Flask backend was configured to run using Gunicorn. A startup command was added inside Azure Stack Settings:

```
gunicorn ecommerce.app:app --bind 0.0.0.0:8000
```

The command binds the application to port 8000, which Azure internally routes to external HTTP requests. During deployment, Azure extracted the application into temporary runtime folders and installed required dependencies from the requirements.txt file.

Several deployment issues were encountered during implementation, including:

- Incorrect module path configuration
- Missing Gunicorn dependency
- Wrong file naming (app.py.py, requirements.txt.txt)
- Startup command mismatch
- Missing package installation

These issues were resolved by correcting folder structures, updating dependencies, and configuring the startup command correctly.

6.4 Dependency Management

The project dependencies were managed using a requirements.txt file. The main packages used include:

flask

pyodbc

gunicorn

- Flask handles backend routing and web application logic.
- PyODBC connects Python Flask with Azure SQL Database.
- Gunicorn acts as the production WSGI server for Azure Linux hosting.

Initially, deployment failures occurred because the requirements.txt file was not placed correctly in the deployment root directory. Azure therefore failed to install required packages automatically.

After moving the file to the correct root path and redeploying the application, dependencies were installed successfully.

6.5 Monitoring and Troubleshooting

Monitoring and troubleshooting played a major role during project deployment. Azure Log Stream displayed real-time runtime logs, while Kudu SSH allowed direct inspection of cloud deployment directories.

Common errors identified during deployment included:

- ModuleNotFoundError: No module named ecommerce
- No module named gunicorn
- Missing package installation
- Incorrect startup command configuration

The Kudu SSH console was used to inspect deployment folders such as:

/home/site/wwwroot

This helped identify misplaced files and duplicate extensions. Log Stream also provided detailed Python stack traces which helped isolate backend startup issues.

6.6 Scalability and Reliability

Azure App Service provides scalability and reliability features that improve application performance under varying workloads. Since the application is hosted on Azure cloud infrastructure, resources can be scaled vertically or horizontally when traffic increases.

Azure SQL Database additionally provides backup management, high availability, and secure cloud storage for structured data. HTTPS communication ensures encrypted data transfer between users and the application.

The cloud deployment model also ensures:

- Remote accessibility
- Centralized database management
- Continuous application availability
- Managed runtime environment
- Automatic infrastructure maintenance

CHAPTER 7 TESTING AND RESULTS

7.1 Testing Methodology

Different types of testing were performed to check whether the e-commerce cloud application works correctly and reliably.

Unit Testing

Individual backend functions and Flask routes were tested separately.

Integration Testing

Testing was done to ensure proper communication between:

- Flask backend
- Azure SQL Database
- Frontend pages

System Testing

The complete application was tested after deployment on Azure to verify that all components work together properly.

Deployment Testing

Cloud deployment testing was done to verify:

- Startup command execution
- Gunicorn server startup
- Dependency installation
- Database connectivity

Monitoring and Debugging

Azure Log Stream and Kudu SSH were used to monitor logs and fix deployment errors.

7.2 Test Cases and Results

ID	Module	Test Description	Expected Result	Actual Result	Status
DB-01	Database	Connect to Azure SQL	Connection successful	Connected successfully	Pass
DB-02	Database	Execute SQL query	Data displayed	Correct data shown	Pass
API-01	Backend	Load Flask route	Route loads properly	Route loaded	Pass
API-02	Backend	Fetch database data	JSON response	Correct response	Pass
UI-01	Frontend	Load homepage	Page opens correctly	Homepage loaded	Pass
UI-02	Frontend	Display data table	Data shown properly	Table displayed	Pass
DEP-01	Deployment	Deploy app to Azure	Successful deployment	Deployment successful	Pass
DEP-02	Deployment	Start Gunicorn server	Server starts correctly	Started successfully	Pass
DEP-03	Deployment	Install dependencies	Packages installed	Installed correctly	Pass
ERR-01	Debugging	Fix module error	Error resolved	Fixed successfully	Pass

ID	Module	Test Description	Expected Result	Actual Result	Status
ERR-02	Debugging	Fix Gunicorn issue	Gunicorn detected	Working properly	Pass
PERF-01	Performance	Load application	Fast response	Loaded quickly	Pass

7.3 Performance Analysis and Inference

The application worked successfully after deployment on Azure.

Observations

- Flask backend connected properly with Azure SQL Database.
- Web pages loaded successfully in the browser.
- SQL queries executed correctly.
- Azure App Service hosted the application without crashes.
- Data retrieval speed was good for small and medium datasets.

Errors Faced During Deployment

Some deployment issues occurred initially:

- ModuleNotFoundError
- Missing Gunicorn package
- Incorrect startup command
- Wrong folder structure
- Dependency installation problems

These issues were fixed by:

- Correcting project structure
- Updating startup command
- Adding Gunicorn to requirements.txt
- Redeploying the application

7.4 User Testing Feedback

The deployed application was tested through browser access.

Verified Features

- Homepage loading
- Database connectivity
- Data display from Azure SQL
- Cloud accessibility

Suggestions for Improvement

Future improvements can include:

- Interactive dashboards
- Better UI design
- Authentication system
- Product filtering

- Real-time analytics

7.5 Security Testing Summary

Basic security testing was performed during deployment.

Security Features Used

- Azure SQL firewall protection
- HTTPS support from Azure App Service
- Secure database connection

Problems Faced

- Firewall initially blocked database access
- Wrong credentials caused connection errors

These problems were fixed by updating:

- Firewall rules
- Database credentials

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENTS

The e-commerce analytics cloud project successfully demonstrated the development and deployment of a cloud-based web application using Microsoft Azure services. The project integrated Flask backend development, Azure SQL Database connectivity, and Azure App Service deployment into a complete working system.

The application was able to:

- Connect with Azure SQL Database
- Retrieve and display e-commerce data
- Deploy successfully on Azure cloud
- Provide browser-based accessibility
- Handle cloud-based database operations

The project also provided practical experience in:

- Cloud deployment
- Azure App Service configuration
- Database connectivity
- Runtime debugging
- Dependency management
- Cloud monitoring

Several deployment issues were faced during implementation, such as missing modules, incorrect startup commands, and dependency installation problems. These issues were resolved using Azure Log Stream and Kudu SSH debugging tools.

The project achieved its main objective of building and deploying a cloud-hosted e-commerce analytics application using Microsoft Azure technologies.

Key Inferences

- Correct project structure is very important in cloud deployment.
- Dependency management plays a major role in successful deployment.
- Azure monitoring tools help quickly identify deployment issues.
- Cloud hosting improves accessibility and scalability.
- Flask and Azure SQL can be integrated effectively for cloud applications.

The project demonstrated how cloud computing technologies can be used to build scalable and accessible web applications for real-world business analytics.

8.2 Limitations

The current system has some limitations:

- The application supports only basic analytics features.
- No authentication or user login system is implemented.

- Interactive dashboards and charts are limited.
- Large datasets may reduce performance.
- Real-time analytics functionality is not available.
- The frontend UI is simple and can be improved further.
- The application depends completely on internet connectivity.

8.3 Future Enhancements

Several improvements can be added in the future to enhance the system:

- Add advanced analytics dashboards and visual charts.
- Implement user authentication and authorization.
- Add product filtering and search functionality.
- Integrate real-time analytics and live updates.
- Deploy frontend and backend as separate cloud services.
- Use Power BI integration for advanced visualization.
- Add AI-based sales prediction and recommendation systems.
- Improve UI/UX using modern frontend frameworks.
- Add API security and token-based authentication.
- Implement automated CI/CD pipelines for deployment automation.

CHAPTER 9 REFERENCES

1. Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2016.
2. Rajkumar Buyya et al., *Cloud Computing: Principles and Paradigms*, Wiley Publications, 2013.
3. Microsoft Azure Documentation, *Microsoft Learn Documentation*, Available at:
<https://learn.microsoft.com/azure>
4. Flask Official Documentation, *Flask Web Framework Documentation*, Available at:
<https://flask.palletsprojects.com>
5. Python Official Documentation, *Python Programming Language Documentation*, Available at:
<https://docs.python.org/3>
6. Microsoft Azure SQL Database Documentation, Available at:
<https://learn.microsoft.com/azure/azure-sql>
7. Unicorn Documentation, *Python WSGI HTTP Server Documentation*, Available at:
<https://unicorn.org>
8. PyODBC Documentation, Available at:
<https://github.com/mkleehammer/pyodbc/wiki>
9. Microsoft Azure App Service Documentation, Available at:
<https://learn.microsoft.com/azure/app-service>
10. Azure Kudu SCM Documentation, Available at:
<https://github.com/projectkudu/kudu/wiki>
11. Microsoft Azure Monitor Documentation, Available at:
<https://learn.microsoft.com/azure/azure-monitor>
12. Thomas Erl, *Cloud Computing: Concepts, Technology & Architecture*, Pearson Education, 2013.
13. Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.
14. OWASP Foundation, *OWASP Top Ten Web Application Security Risks*, 2021. Available at:
<https://owasp.org/www-project-top-ten>
15. GitHub Documentation, *GitHub Actions Documentation*, Available at:
<https://docs.github.com/actions>
 - a.  Docker Documentation, *Docker Official Documentation*, Available at:
<https://docs.docker.com>

CHAPTER A

Appendices

A.1 Sample API Endpoints

Method	Endpoint	Description
GET	/	Home page
GET	/data	Fetch e-commerce data
GET	/dashboard	Display dashboard
GET	/testdb	Test database connection

A.2 Deployment Architecture

User Browser



Azure App Service



Flask Backend



Azure SQL Database

A.3 Technologies Used

- Python
- Flask
- Azure App Service
- Azure SQL Database
- PyODBC
- Gunicorn
- HTML/CSS

- **GitHub**
-

A.4 Project Timeline

Week Activity

1–2 Planning and Design

3–5 Development

6–7 Azure Deployment

8 Testing and Debugging

9 Documentation

A.5 Deployment Issues Faced

- **Incorrect startup command**
- **Missing Gunicorn package**
- **Module import errors**
- **Azure SQL firewall issue**

These issues were fixed using Azure Log Stream and Kudu SSH console.